

Metaheuristic-Based Navigation Optimization for Heterogeneous Autonomous Vehicle Fleets

Seifeldin Abbas¹, Rasheed Atia¹, Yassin Otifa¹, Jessica Magdy^{2,3} and Omar M. Shehata^{2,3}

¹ Computer Science Department, Faculty of Media Engineering and Technology, German University in Cairo, Egypt

² Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt

³ Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

{seifeldin.hassan, rasheed.atia, yassin.otifa}@student.guc.edu.eg, {jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—The Heterogeneous Fleet Vehicle Routing Problem (HFVRP) represents a complex combinatorial optimization challenge with significant real-world applications in logistics and supply chain management. This paper presents a comparative study of four metaheuristic algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), Ant Colony Optimization (ACO), and Teaching-Learning Based Optimization (TLBO)—applied to the HFVRP. The problem is modeled with strict hard constraints on vehicle capacity and maximum travel distance, prohibiting any infeasible solutions during the search process. A "Best Insertion" heuristic is implemented to generate feasible initial solutions. The algorithms are evaluated on a set of seven test cases of increasing complexity, ranging from simple 4-customer scenarios to a large-scale 60-customer challenge. Performance is analyzed based on solution quality (final cost), execution time, and convergence behavior. Results indicate that while population-based methods like ACO and TLBO offer robust search capabilities, the single-point Simulated Annealing algorithm demonstrates superior efficiency and solution quality for the specific constraints and scale of the tested instances.

Index Terms—HFVRP, Simulated Annealing, Genetic Algorithm, Ant Colony Optimization, TLBO, Metaheuristics.

I. INTRODUCTION

THE Vehicle Routing Problem (VRP) is a fundamental problem in operations research and logistics, involving the design of optimal delivery routes for a fleet of vehicles to serve a set of customers. The Heterogeneous Fleet Vehicle Routing Problem (HFVRP) extends the classical VRP by considering a fleet of vehicles with varying capacities, costs, and travel limits. This variant more accurately reflects real-world scenarios where logistics companies operate mixed fleets of trucks, vans, and motorcycles.

The HFVRP has been extensively studied over the past decades [6], and while advanced exact methods like branch-cut-and-price have been developed [7], the problem remains computationally intractable for large-scale instances. Consequently, metaheuristic algorithms are widely used to find high-quality near-optimal solutions within reasonable time frames. This paper focuses on solving a constrained version of the HFVRP where vehicles have distinct capacities and maximum travel distances. We implement and compare four distinct metaheuristic approaches:

- 1) **Simulated Annealing (SA)**: A trajectory-based probabilistic technique inspired by thermodynamics.
- 2) **Genetic Algorithm (GA)**: A population-based evolutionary algorithm inspired by natural selection.
- 3) **Ant Colony Optimization (ACO)**: A swarm intelligence method based on the foraging behavior of ants.
- 4) **Teaching-Learning Based Optimization (TLBO)**: A population-based algorithm inspired by the influence of a teacher on learners in a class.

The study enforces a "hard constraint" model, ensuring that every solution generated or explored remains strictly feasible, which presents a significant challenge for initialization and neighbor generation.

II. STATE OF ART

The Vehicle Routing Problem (VRP) has evolved significantly since its inception, with the Heterogeneous Fleet VRP (HFVRP) garnering substantial attention due to its applicability in modern logistics. Recent literature emphasizes the use of metaheuristics to tackle the NP-hard nature of these problems, often employing hybrid heuristics to address a broad class of routing variants [8]. Beyond traditional logistics, these optimization techniques are increasingly vital for complex path planning and task allocation in diverse domains, such as multi-AGV search and rescue missions [10] and sustainable energy layout optimization [11].

A. Related Work

Phuc and Thao (2021) explored the complexity of e-commerce logistics, utilizing an Ant Colony Optimization (ACO) algorithm for a VRP variant with time windows and heterogeneous fleets. Their model demonstrated high reliability for large-scale instances [1]. Similarly, optimization for Last Mile Delivery (LMD) in dense urban environments has been approached using Large Neighborhood Search (LNS) metaheuristics. Research indicated that optimizing routes could reduce operational costs by over 58%, often proving more effective than simply varying fleet composition [2].

In the domain of emergency logistics, Xu et al. (2022) applied a Tabu Search (TS) algorithm to manage evacuation

scenarios, achieving near-optimal results significantly faster than exact CPLEX solvers [3]. This highlights the necessity of metaheuristics when response time is critical.

More recently, focus has shifted towards customer-centric variants. Stavropoulou (2022) proposed a "Consistent" VRP (HConVRP) to ensure service consistency, using a hierarchical approach to balance cost efficiency with customer satisfaction [4]. Additionally, Máximo et al. (2022) developed an Adaptive Iterated Local Search (AILS) for the classical HFVRP, using dynamic perturbation mechanisms to escape local optima [5].

B. Experimental Scope

Despite the abundance of metaheuristic algorithms that exist in the literature, this study focuses on four representative and widely adopted methods spanning both trajectory-based and population-based paradigms. This selection enables a controlled and interpretable comparison across fundamentally different search strategies.

The primary contribution of this work lies not in proposing a novel metaheuristic, but in providing a systematic comparative analysis under strict hard-constraint feasibility, a setting that is less explored in existing HFVRP literature.

III. PROBLEM FORMULATION

The HFVRP is modeled on a complete graph with a central depot (node 0) and a set of customers $C = \{1, \dots, n\}$. A heterogeneous fleet V is available, where each vehicle $v \in V$ has a capacity Q_v and a maximum travel distance D_v . Let x_{ijv} be a binary variable equal to 1 if vehicle v travels from node i to j , and y_v be a binary variable equal to 1 if vehicle v is used. Demand for customer i is q_i , and d_{ij} is the symmetric Euclidean distance between i and j .

We assume all routes start and end at the depot, each customer's deterministic demand is fully satisfied by a single visit, and no time windows are imposed. The objective is to minimize a weighted sum of total distance and vehicles used ($F = w_1 f_1 + w_2 f_2$, where $w_1 \gg w_2$ to emphasize distance reduction):

$$\min \sum_{v \in V} \sum_i \sum_{j \neq i} w_1 d_{ij} x_{ijv} + \sum_{v \in V} w_2 y_v \quad (1)$$

Subject to:

$$\sum_{v \in V} \sum_{i \neq j} x_{ijv} = 1, \quad \forall j \in C \quad (2)$$

$$\sum_{i \in C} q_i \sum_{j \neq i} x_{ijv} \leq Q_v, \quad \forall v \in V \quad (3)$$

$$\sum_i \sum_{j \neq i} d_{ij} x_{ijv} \leq D_v, \quad \forall v \in V \quad (4)$$

Equations (2), (3), and (4) ensure exactly one visit per customer, adherence to strict vehicle capacities, and maximum distance limits, respectively.

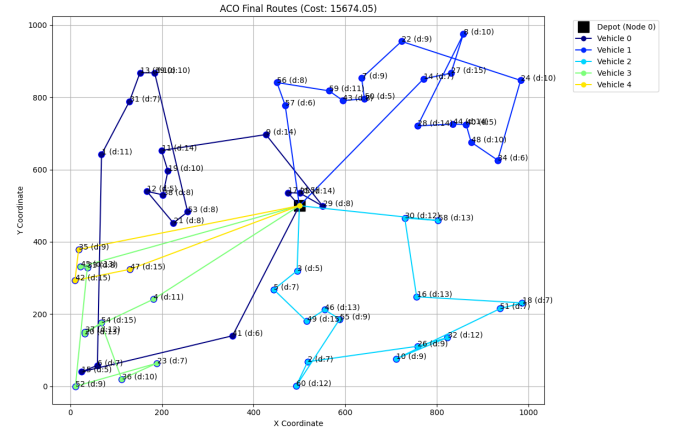


Fig. 1. Illustrative representation of the spatial distribution of customer nodes, demands, and the central depot.

IV. ALGORITHM IMPLEMENTATION

A. Initialization Strategy

To satisfy the strict hard constraints, a Randomized Constructive Heuristic was implemented. This ensures that every algorithm starts with a diverse set of 100% feasible solutions.

B. Mapping Representation to Physical Space

The numerical representation (genotype) translates directly to the physical routing map (phenotype) through a specific decoding sequence. Given a route $r_k = [c_1, c_2, \dots, c_L]$ for Vehicle k :

- 1) **Start:** The vehicle departs from the central Depot (Node 0) located at coordinate (X_{depot}, Y_{depot}) .
- 2) **Traversal:** The vehicle travels in Euclidean space to the coordinate of the first customer $c_1(x_1, y_1)$, then to $c_2(x_2, y_2)$, and continues sequentially through the list.
- 3) **Return:** After visiting the final customer c_L , the vehicle returns directly to the Depot (Node 0).

The total Euclidean distance for this specific route on the map is calculated as:

$$D_{route} = d(0, c_1) + \sum_{i=1}^{L-1} d(c_i, c_{i+1}) + d(c_L, 0) \quad (5)$$

This explicit "Depot-to-Depot" loop is visually represented in the result figures, where all colored route lines originate and terminate at the central black square. Empty lists in the solution representation (e.g., $r_4 = []$) correspond to vehicles that remain stationary at the depot, incurring zero distance cost.

C. Neighborhood Operators

To navigate the complex solution space, three specific neighborhood operators were implemented. These operators are applied stochastically to transform a feasible solution S into a neighbor solution S' .

Algorithm 1 Randomized Constructive Heuristic

```
1: Input: Customers  $C$ , Vehicles  $V$ 
2: Output: Feasible Solution  $S$ 
3: Shuffle Customer List  $C_{rand}$ 
4: for each customer  $c$  in  $C_{rand}$  do
5:    $BestCost \leftarrow \infty, BestPos \leftarrow \emptyset$ 
6:   for each vehicle  $v$  in  $V$  do
7:     for each position  $p$  in Route  $v$  do
8:       if Insertion is Feasible (Capacity & Distance) then
9:         Calculate  $\Delta Cost$ 
10:        if  $\Delta Cost < BestCost$  then
11:           $BestCost \leftarrow \Delta Cost$ 
12:           $BestPos \leftarrow (v, p)$ 
13:        end if
14:      end if
15:    end for
16:  end for
17:  if  $BestPos \neq \emptyset$  then
18:    Insert  $c$  at  $BestPos$  in  $S$ 
19:  else
20:    Return Infeasible (Restart)
21:  end if
22: end for
23: return  $S$ 
```

1) *Inter-Route Swap*: Two customers, $u \in r_i$ and $v \in r_j$, are selected. The new routes are formed by exchanging their positions. This operator is primary for **load balancing** between vehicles.

$$S' = S \setminus \{u \in r_i, v \in r_j\} \cup \{v \in r_i, u \in r_j\} \quad (6)$$

2) *Relocate (Shift)*: A customer u is removed from its current route r_i and inserted into a different route r_j (or a different position in r_i). This operator effectively reduces the number of active vehicles by emptying underutilized routes.

3) *2-Opt (Intra-Route Optimization)*: A local search operator is applied within a single route. Two positions i and j are selected, and the sequence of customers between them is reversed. This operation removes route crossings and improves route compactness, often reducing the total travel distance without altering vehicle assignments.

D. Simulated Annealing (SA)

The SA algorithm uses a geometric cooling schedule. The neighbor generation logic is critical for maintaining feasibility.

E. Genetic Algorithm (GA)

The GA uses a heuristic crossover to combine routes while maintaining feasibility.

F. Teaching-Learning Based Optimization (TLBO)

TLBO was implemented with a population size of 50. In the Teacher Phase, the mean solution moves towards the best solution (Teacher). In the Learner Phase, solutions interact randomly. The "movement" is modeled using the BCRC

Algorithm 2 Simulated Annealing

```
1:  $S \leftarrow \text{GenerateInitialSolution}()$ 
2:  $T \leftarrow T_{init}$ 
3: while  $T > T_{final}$  do
4:   for  $i = 1$  to  $MaxIter$  do
5:      $S' \leftarrow \text{GetNeighbor}(S)$  (Swap, Move, 2-Opt)
6:     if  $\text{CheckFeasibility}(S')$  is True then
7:        $\Delta E \leftarrow \text{Cost}(S') - \text{Cost}(S)$ 
8:       if  $\Delta E < 0$  or  $\text{rand}(0, 1) < e^{-\Delta E/T}$  then
9:          $S \leftarrow S'$ 
10:      end if
11:    end if
12:  end for
13:   $T \leftarrow T \times \alpha$ 
14: end while
```

Algorithm 3 Genetic Algorithm

```
1:  $Pop \leftarrow \text{InitializePopulation}(N_{pop})$ 
2: for  $gen = 1$  to  $MaxGen$  do
3:    $\text{EvaluateFitness}(Pop)$ 
4:    $NewPop \leftarrow \text{Elitism}(Pop)$ 
5:   while  $|NewPop| < N_{pop}$  do
6:      $P1, P2 \leftarrow \text{TournamentSelection}(Pop)$ 
7:      $Child \leftarrow \text{BestCostRouteCrossover}(P1, P2)$ 
8:      $Child \leftarrow \text{Repair}(Child)$  (using Best Insertion)
9:     if  $\text{rand}() < p_m$  then
10:       $Child \leftarrow \text{Mutate}(Child)$ 
11:    end if
12:    Add  $Child$  to  $NewPop$ 
13:  end while
14:   $Pop \leftarrow NewPop$ 
15: end for
```

crossover operator to ensure that the resulting student solutions remain valid within the hard constraints of the HFVRP.

G. Parameter Configuration

The performance of metaheuristic algorithms is highly sensitive to hyperparameter tuning. Parameters were standardized across algorithms where possible (e.g., population size). The specific configurations used for the final computational experiments are detailed in Table I.

A population size of 50 was selected for GA, ACO, and TLBO to balance exploration diversity with computational overhead. For SA, the cooling schedule (from $T_{init} = 1000$ to $T_{final} = 1.0$ with $\alpha = 0.933$ and 75 iterations per temperature level) yielding approximately 100 temperature reductions, translating to roughly 7500 evaluations. The cooling schedule was calibrated such that the total number of fitness evaluations matches the other population-based algorithms, ensuring a fair comparison.

Similarly, 7,500 evaluations are performed by GA and ACO (50 individuals $\times$ 150 generations $\approx$ 7500 evaluations), and TLBO (50 individuals $\times$ 75 iterations $\times$ 2 evaluations per iteration $\approx$ 7500 evaluations).

TABLE I
ALGORITHM HYPERPARAMETERS

Algorithm	Parameter	Value
Simulated Annealing	Initial Temperature (T_{init})	1000
	Final Temperature (T_{final})	1.0
	Cooling Rate (α)	0.933
	Iterations per Temp	75
Genetic Algorithm	Population Size	50
	Generations	150
	Crossover Rate	0.8
	Mutation Rate	0.3
Ant Colony Opt.	Number of Ants	50
	Iterations	150
	Alpha (Pheromone Weight)	1.0
	Beta (Heuristic Weight)	3.0
TLBO	Population Size	50
	Iterations	75
	Evaluations per Iteration	2

H. Solution Representation

To explicitly model the heterogeneous fleet, a **Direct Route Encoding** (or "List of Lists") was utilized. Unlike indirect permutation encodings that require delimiters, this structure maps directly to the physical problem.

Let S be the solution set $S = \{r_1, r_2, \dots, r_m\}$, where m is the number of vehicles in the fleet. Each element r_k represents the specific route for vehicle k and is defined as an ordered sequence of customer indices:

$$r_k = [c_{k,1}, c_{k,2}, \dots, c_{k,L_k}] \quad (7)$$

where L_k is the number of customers served by vehicle k . An empty list $r_k = \emptyset$ indicates that vehicle k is unused.

This representation allows for $O(1)$ access to any vehicle's route, which is critical for the "Hard Constraint" model. When checking feasibility, the algorithm can calculate the total demand (D_{load}) and total distance (D_{dist}) for a specific vehicle r_k independently:

$$D_{load}(r_k) = \sum_{i=1}^{L_k} q_{c_{k,i}} \leq Q_k \quad (8)$$

This direct mapping significantly reduces the overhead of decoding and ensures that constraint violations can be identified and penalized immediately during the neighbor generation process.

1) *Illustrative Example of Encoding:* To clarify the "List of Lists" structure, consider a small instance with 5 customers ($C = \{1, 2, 3, 4, 5\}$) and a heterogeneous fleet of 3 vehicles ($V = \{V_0, V_1, V_2\}$). A possible valid solution S would be represented in memory as a 2D jagged array (or list of lists):

$$S = [[1, 3], [5, 2, 4], []] \quad (9)$$

This encoding translates to the following physical routes:

- **Vehicle 0 (List 0):** Starts at Depot → Visits Customer 1 → Visits Customer 3 → Returns to Depot.
- **Vehicle 1 (List 1):** Starts at Depot → Visits Customer 5 → Visits Customer 2 → Visits Customer 4 → Returns to Depot.

- **Vehicle 2 (List 2):** The empty list $[\]$ indicates that Vehicle 2 is **unused** and remains at the depot (Cost = 0).

This direct mapping allows the algorithm to trivially calculate the load for Vehicle 1 by summing demands $q_5 + q_2 + q_4$ and checking it against Capacity Q_1 , independent of the other vehicles.

V. RESULTS AND DISCUSSION

The algorithms were rigorously evaluated on a diverse set of seven test cases, ranging from simple proof-of-concept scenarios to large-scale optimization challenges. To ensure scientific rigor and account for the stochastic nature of meta-heuristics, a statistical analysis was performed on the two most complex cases (Case 6 and Case 7). This involved executing 10 independent runs for each algorithm to gather data on stability and repeatability.

A. Performance Indices

The comparison framework is built upon four key performance indicators (KPIs) to provide a holistic view of algorithmic efficiency: **1) Best Cost (Optimality):** The absolute minimum objective function value found across all independent runs, indicating peak performance capability. **2) Mean Cost (Average Performance):** The arithmetic mean of the final costs over 10 runs, providing a realistic expectation of typical performance. **3) Standard Deviation (Stability):** A measure of solution variability; a lower standard deviation implies a reliable algorithm less sensitive to random initialization. **4) Execution Time (Efficiency):** The average wall-clock time required to complete a single optimization run, measured in seconds.

B. Statistical Analysis: Case 6 (Medium)

Test Case 6, "The Medium Labyrinth," features 25 customers distributed in clustered formations, served by a heterogeneous fleet of 4 vehicles. This case tests the algorithms' ability to navigate local optima created by the clusters.

The statistical results for Case 6 are presented in Table II. Additionally, Figure 3 shows the convergence graph, where the x-axis shows cumulative fitness evaluations, ensuring a fair comparison of computational effort across all algorithms.

TABLE II
STATISTICAL ANALYSIS (10 RUNS) - CASE 6

Algo	Best Cost	Mean Cost	Std Dev	Time(s)
SA	3593.45	3809.14	351.77	0.57
GA	3593.45	3623.95	65.00	0.66
ACO	3610.40	3607.78	8.68	12.14
TLBO	3604.87	3648.56	88.49	0.44

In this medium-complexity scenario, the results are highly competitive. Simulated Annealing (SA) and the Genetic Algorithm (GA) tied for the best-found solution (3593.45), demonstrating that both trajectory-based and population-based methods can locate the global optimum in this search space.

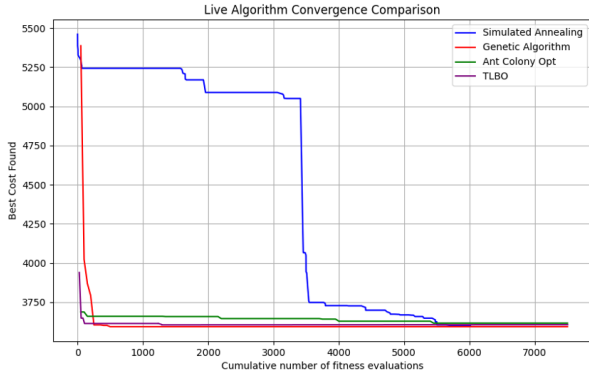


Fig. 2. Convergence Comparison for Test Case 6 (25 Customers)

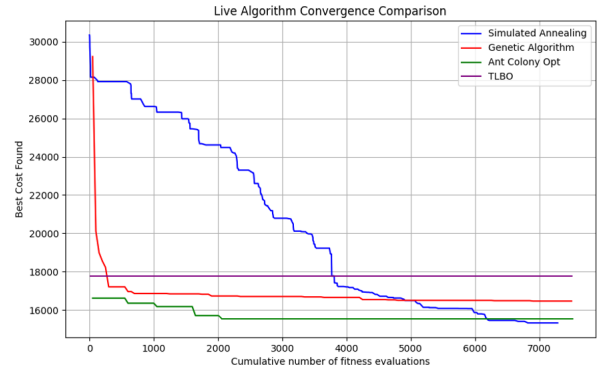


Fig. 3. Convergence Comparison for Test Case 7 (60 Customers)

However, Ant Colony Optimization (ACO) demonstrated superior stability, with a negligible standard deviation of 8.68. This indicates that while ACO is significantly slower (12.14s compared to TLBO's 0.44s), it reliably converges to a near-optimal solution in almost every run.

C. Statistical Analysis: Case 7 (Large)

The "60-Customer Challenge," is a stress test designed to differentiate the algorithms' scalability. It involves 60 randomly distributed customers and a larger fleet of 7 vehicles. However, SA's stability is highly problem-dependent; for example, it achieved the best standard deviation in recent UGV waste collection studies [9].

TABLE III
STATISTICAL ANALYSIS (10 RUNS) - CASE 7

Algo	Best Cost	Mean Cost	Std Dev	Time(s)
SA	15323.91	16217.15	733.31	0.96
GA	16463.94	16833.11	718.33	2.90
ACO	15657.85	15824.67	236.08	67.66
TLBO	17794.96	17056.29	375.71	1.38

Visual inspection of Case 7 solutions (Figure 3) reveals distinct structural differences. **Simulated Annealing** produces highly compact, radially fanned routes indicative of near-optimal VRP solutions, finding the absolute best cost (15,323.91) with extreme speed (0.96s). However, its high standard deviation (733) requires multiple restarts to ensure reliability. In contrast, **GA and TLBO** display crossing routes and overlapping sectors; their BCRC crossover and "Best Insertion" repair mechanisms struggle to unlink distant customers in highly constrained spaces. **ACO** proved the most reliable on average (lowest Mean Cost and a Std Dev of 236), but its massive computational overhead (67s vs. 1s for SA) reduces its viability for real-time applications.

D. Sensitivity Analysis of Hyperparameters

The performance of the implemented algorithms is heavily dependent on the tuning of hyperparameters. We conducted a qualitative sensitivity analysis to understand their impact.

1) *Impact of Cooling Schedule (SA)*: In Simulated Annealing, the cooling rate α determines the speed of convergence. Our experiments with $\alpha = 0.8$ resulted in premature convergence (getting stuck in local optima), while $\alpha = 0.99$ excessively prolonged the execution time without significant quality gains. The chosen value of $\alpha = 0.95$ provided the optimal balance, allowing the algorithm sufficient time to explore high-energy states before settling into a minimum.

2) *Impact of Population Size (GA & TLBO)*: For the population-based methods, the population size dictates the genetic diversity. Small populations ($N < 20$) led to rapid "genetic drift," where the population became homogeneous too quickly, halting progress. Large populations ($N > 100$) increased the computational cost linearly without a proportional improvement in solution quality. A population of 50 was found to maintain enough diversity to support the crossover operators while keeping runtime within acceptable limits.

3) *Impact of Pheromone Influence (ACO)*: In ACO, the balance between pheromone history (α) and heuristic information (β) is crucial. We observed that setting $\alpha > \beta$ caused the ants to essentially "follow the leader," rapidly converging to the first found solution. Conversely, $\beta \gg \alpha$ turned the algorithm into a greedy search. Setting $\beta = 3$ and $\alpha = 1$ encouraged the ants to prioritize short local distances (greedy) initially, while gradually building pheromone trails to identify globally superior paths.

E. Comparative Analysis of Efficiency vs. Optimality

To visualize the trade-off between solution quality and computational effort, Figure 4 presents a normalized comparison of the best costs and execution times for Case 7.

The bar chart highlights the dominance of Simulated Annealing in terms of raw optimality for this specific problem structure. While ACO remains a strong contender, the "Best Insertion" heuristic used in GA and TLBO appears to limit their ability to refine solutions as effectively as SA's trajectory-based approach.

F. Computational Complexity Analysis

The theoretical complexity of the algorithms offers insight into the runtime discrepancies observed in Table III.

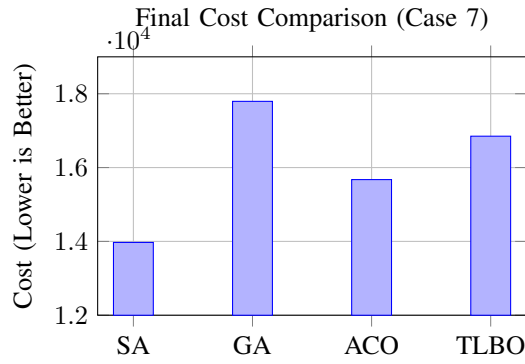


Fig. 4. Best Cost Comparison for Case 7. SA achieves the lowest cost.

- **Simulated Annealing:** The complexity is dominated by the neighbor generation and cost calculation, $O(k \cdot n)$, where n is the number of customers and k is the number of iterations. Since neighbor evaluation is $O(1)$ (incremental update), SA is extremely fast.
- **Genetic Algorithm / TLBO:** These are $O(G \cdot P \cdot n^2)$ where G is generations and P is population size. The n^2 term arises from the "Best Insertion" repair operator, which must scan all route positions for every unassigned customer. This explains why both are slower than SA.
- **Ant Colony Optimization:** This is the most expensive, with complexity $O(I \cdot A \cdot n^2)$, where I is iterations and A is ants. For every step of every ant, the algorithm must calculate probabilities for all remaining customers, leading to the high execution times observed (67s).

G. Search Space Complexity Analysis

The necessity of using metaheuristics is justified by the combinatorial explosion of the HFVRP search space. For a problem with N customers and M vehicles, the number of possible routing configurations approximates $(N + M - 1)!$. For our largest instance (Case 7, $N = 60, M = 7$), the solution space size is proportional to $66!$, a number exceeding 10^{90} . Exact methods would fail to explore even a fraction of this space. The comparative results demonstrate that while the search space is vast, the "Hard Constraint" formulation effectively prunes the infeasible regions, allowing algorithms like Simulated Annealing to identify high-quality "islands" of feasibility without wasting computational resources on invalid permutations.

VI. CONCLUSION AND FUTURE WORK

This study implements and compares four metaheuristic algorithms for the HFVRP under strict feasibility constraints. The statistical analysis of 10 independent runs yielded conclusive insights:

- **Simulated Annealing** is the "High Risk, High Reward" choice. It is the fastest algorithm and capable of finding the absolute global optimum, but it requires multiple runs due to its stochastic instability.

- **Ant Colony Optimization** is the "Reliable" choice. It consistently produces high-quality solutions with low variance, making it ideal when stability is preferred over speed. However, it is computationally expensive (approx. 60x slower than SA).
- **GA and TLBO** provided balanced performance but were outperformed in the largest test case, suggesting that the "Best Insertion" repair mechanism used in their crossover operators may limit exploration in highly constrained spaces.

Future Research could focus on hybridizing SA and ACO to combine SA's speed with ACO's stability.

Additionally, investigating "Soft Constraint" models with dynamic penalty functions could allow the algorithms to traverse infeasible regions and find better global optima.

Moreover, a Reinforcement Learning (RL) approach could be employed where a Deep Q-Network (DQN) or Proximal Policy Optimization (PPO) agent could be trained on small instances to learn generalizable routing heuristics, potentially replacing the greedy "Best Insertion" logic with a learned policy that anticipates future congestion.

Lastly, the test cases used in this study are synthetically generated to provide greater flexibility in isolating the impact of extreme hard constraints. Future work will extend the evaluation to publicly available benchmark datasets to enhance reproducibility and comparability.

REFERENCES

- [1] P. N. Ky Phuc and N. L. Phuong Thao, "Ant colony optimization for multiple pickup and multiple delivery vehicle routing problem with time window and heterogeneous fleets," *Logistics*, vol. 5, no. 2, p. 28, 2021.
- [2] T. Ayu et al., "Optimizing the heterogeneous fleet vehicle routing problem with time window on urban last mile delivery," in *IOP Conference Series: Earth and Environmental Science*, vol. 830, no. 1. IOP Publishing, 2021, p. 012100.
- [3] L. Xu, Z. Wang, X. Chen, and Z. Lin, "Multi-parking lot and shelter heterogeneous vehicle routing problem with split pickup under emergencies," *IEEE Access*, vol. 10, pp. 1-1, 01 2022.
- [4] F. Stavropoulou, "The consistent vehicle routing problem with heterogeneous fleet," *Computers Operations Research*, vol. 140, p. 105644, 2022.
- [5] V. R. Máximo, J.-F. Cordeau, and M. C. Nascimento, "An adaptive iterated local search heuristic for the heterogeneous fleet vehicle routing problem," *Computers Operations Research*, vol. 148, p. 105954, 2022.
- [6] Ç. Koç, T. Bektas, O. Jabali, and G. Laporte, "Thirty years of heterogeneous vehicle routing," *European Journal of Operational Research*, 2016.
- [7] A. Pessoa, R. Sadykov, and E. Uchoa, "Enhanced Branch-Cut-and-Price algorithm for heterogeneous fleet VRPs," *European Journal of Operational Research*, 2018.
- [8] P. H. V. Penna, A. Subramanian, L. S. Ochi, T. Vidal, and C. Prins, "A hybrid heuristic for a broad class of HFVRPs," *Annals of Operations Research*, 2019.
- [9] F. Ahmed et al., "Route to Clean Cities: Waste Collection Optimization with UGVs," in *2025 15th International Conference on Electrical Engineering (ICEENG)*, IEEE, May 2025, pp. 1-6.
- [10] A. H. Ghoneim et al., "Metaheuristic Optimization for Multi-AGV Path Planning in Search and Rescue Missions," in *2025 15th International Conference on Electrical Engineering (ICEENG)*, IEEE, May 2025, pp. 1-6.
- [11] M. Elsheikh, A. Yonis, J. Magdy, and O. M. Shehata, "Advancing Sustainable Energy: Analyzing Stochastic Algorithms and Q-Learning for Wind Farm Layout Optimization," in *2024 5th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*, Cairo, Egypt, 2024, pp. 128-132.